

SeuGado — Leandro: fazenda, lotes, plano, ciclo e deploy

Login e cadastro da fazenda e dos lotes, página do plano, ciclo semanal e publicação
27/09/2026

Sua parte em uma frase: você liga o sistema de ponta a ponta. O produtor entra, cadastra a fazenda e o rebanho, vê o plano da semana, e o sistema roda sozinho toda segunda, publicado na internet.

Você entrega:

- 1. Identidade visual (Figma) + `tema.css`
- 2. Login
- 3. Fazenda: onboarding e configurações (funcionários, dias de manejo, Telegram)
- 4. Lotes: cadastro por categoria
- 5. Página “Plano da semana”
- 6. Ciclo semanal (`jobs/ciclo.py`)
- 7. Deploy: Render, Vercel e GitHub Actions

Prazo: a L0 (`/me`) entra na `main` até as 10h de domingo, porque o Ezequiel depende dela; o resto em PR até domingo à noite. A ordem das tarefas abaixo é a ordem de prioridade: se o tempo apertar, o deploy (L7) pode ser feito na segunda com a equipe.

1. O SeuGado em um minuto

O pecuarista divide o pasto em **piquetes** e o gado em **lotes**. O capim tem uma altura certa para o lote entrar e uma altura em que ele precisa sair. O SeuGado estima a altura de cada piquete por satélite e clima e manda, **toda semana**, um plano pelo Telegram: “segunda: mova o lote Recria do Piquete 2 para o Piquete 1”. O plano respeita a rotina da fazenda: só propõe movimentação nos **dias de manejo** que o produtor escolheu e nunca mais movimentações por dia do que os funcionários dão conta.

Palavras que você vai usar

Termo	O que é
Lote	Grupo de animais que anda junto. É a unidade de manejo
Categoria	bezerro (até ~1 ano), novilho (recria, ~1 a 3 anos), adulto (vaca, boi, touro)
Composição do lote	Quantos animais de cada categoria e o peso médio de cada uma
Peso médio	Define quanto o lote come ($\approx 2,2\text{--}2,4\%$ do peso por dia). Se o produtor não sabe, o sistema estima pela tabela de Unidade Animal (bezerro 112,5 kg · novilho 337,5 kg · adulto 450 kg) e a recomendação fica com confiança menor
Lote indissolúvel	Lote que nunca pode ser juntado a outro (ex.: vacas com bezerro ao pé)

Termo	O que é
Dias de manejo preferidos	Dias da semana em que a fazenda aceita mexer no gado. Guardados como números: 0 = segunda ... 6 = domingo
Manejes por funcionário por dia	Quantas movimentações de lote cada funcionário consegue fazer num dia
Evento	Registro imutável de algo que aconteceu ("lote criado"). Nunca se edita o passado: grava-se um evento novo
Ciclo	A rotina que coleta satélite, estima o pasto, gera o plano e manda pelo Telegram

Por que não pedimos raça, sexo nem brinco? O que muda o consumo de capim é o **peso** e a **idade** do animal, não a raça nem o sexo. Cadastro animal por animal é outro produto (ERP) e está fora do escopo (ADR-001). O produtor informa só "40 novilhos de ~340 kg".

Entradas e saídas: o que você recebe, de quem, e o que entrega

Esta seção é o seu contrato com o resto da equipe. **Nomes, tipos e formatos são exatamente estes**; se algo aqui parecer faltar ou estar errado, fale com o Kauê antes de inventar outro formato.

Resumo

	O quê	De quem / para quem	Como chega / sai
Recebe	login e sessão	Supabase Auth, via <code>api/auth.py</code> do Kauê	<code>Usuario</code> , <code>FazendaAutorizada</code> ; no front, <code>supabase</code> e <code>useFazenda()</code>
Recebe	lista de piquetes	Ezequiel	<code>GET /fazendas/{id}/piquetes</code> → <code>Piquete[]</code> (<code>id</code> , <code>nome</code> , <code>situacao</code> , <code>lote_atual_id</code> ...)
Recebe	peso de reserva por categoria	Kauê	<code>peso_por_ua_kg(categoria: CategoriaAnimal) -> float</code> (<code>planner/estado.py</code>)
Recebe	funções do ciclo	Kauê, João, Leo	assinaturas na seção 5.2
Recebe	o plano atual	João	<code>carregar_plano_atual(conn, fazenda_id) -> PlanoManejo None</code>
Entrega	Fazenda (rotina)	João (via ciclo)	<code>carregar_fazenda(conn, fazenda_id) -> Fazenda</code>
Entrega	usuário + fazenda para o site todo	todas as páginas (inclusive o Mapa do Ezequiel)	<code>GET /me</code> → <code>{"usuario_id", "email", "fazenda": Fazenda null}</code>
Entrega	lotes gravados como eventos	banco → Kauê (consumo e posição) → João	<code>lote_criado</code> , <code>lote_alterado</code> , <code>lote_dissolvido</code> , <code>manejo_confirmado</code>
Entrega	plano atual para a web	Ezequiel (mapa) e a sua página Plano	<code>GET /fazendas/{id}/plano/atual</code> → <code>plano_para_dict(plano)</code> ou 404
Entrega	recálculo	Leo	<code>executar_ciclo(conn, fazenda_id, hoje=None, ingerir_satelite=True, enviar=True, atualizado=False) -> PlanoManejo</code>

	O quê	De quem / para quem	Como chega / sai
Entrega	ciclo semanal automático	todos	GitHub Actions → <code>python -m seugado.jobs.ciclo --todas</code>

Fazenda na API (JSON) e no Python

Fazenda: a configuração (de `seugado.core.models`)

Campo	Tipo	Valores	Significado
<code>id</code>	<code>UUID</code>	—	Fazenda
<code>nome</code>	<code>str</code>	—	Nome
<code>timezone</code>	<code>str</code>	"America/Fortaleza"	Fuso
<code>funcionarios_disponiveis</code>	<code>int</code>	≥ 1	Pessoas que fazem o manejo
<code>manejos_por_funcionario_dia</code>	<code>int</code>	≥ 1	Movimentações por pessoa por dia
<code>dias_preferenciais_manejo</code>	<code>tuple[int, ...]</code>	0 = segunda ... 6 = domingo	Dias em que se pode mover gado
<code>ativo</code>	<code>bool</code>	—	Fazenda ativa

Em JSON (GET `/me`, POST `/fazendas`, PUT), os mesmos campos, sem `ativo`; `dias_preferenciais_manejo` vira lista (`[0, 3]`) e `id` vira texto.

Lote que você devolve (JSON), campo a campo

Campo	Tipo	Exemplo	Origem
<code>id</code>	texto (UUID)	"2222...0001"	<code>estado_lote.lote_id</code>
<code>nome</code>	texto	"Recria"	<code>estado_lote.nome</code>
<code>indissoluvel</code>	booleano	false	<code>estado_lote.indissoluvel</code>
<code>composicao</code>	lista de {categoria, n_animais, peso_medio_kg, origem_peso}	[{"categoria": "novilho", "n_animais": 150, "peso_medio_kg": 337.5, "origem_peso": "ua_tabela"}]	<code>estado_lote.composicao</code> (JSON)
<code>piquete_atual_id / piquete_atual_nome</code>	texto ou null	"...", "Piquete 2"	<code>estado_lote.piquete_atual_id + estado_piquete.nome</code>
<code>desde</code>	texto ISO ou null	"2026-09-25"	<code>estado_lote.desde</code>
<code>peso_vivo_total_kg</code>	número	50625.0	<code>estado_lote.peso_vivo_total_kg</code>
<code>n_animais_total</code>	número	150	soma de <code>n_animais</code>

categoria: "bezerro" | "novilho" | "adulto". origem_peso: "produtor" (informado) | "ua_tabela" (estimado: 112,5 / 337,5 / 450 kg).

O plano que você mostra

PlanoManejo: o plano da semana

Tipo Python em `seugado.contratos`; em JSON via `plano_para_dict` / `plano_de_dict`; em TypeScript em `frontend/src/lib/tipos.ts`. Exemplo completo: `tests/fixtures/plano_exemplo.json`.

Campo	Tipo (Python)	Em JSON	Significado
id	UUID	texto	Id do plano
fazenda_id	UUID	texto	Fazenda dona do plano
data_geracao	datetime (UTC, com fuso)	"2026-09-28T08:00:00+00:00"	Quando foi gerado
data_inicio	date	"2026-09-28"	Primeiro dia do plano (hoje)
horizonte_dias	int	7	Sempre 7
movimentacoes	tuple[Movimentacao, ...]	lista	Ordenadas por (data, lote_nome)
alertas	tuple[Alerta, ...]	lista	Ordenados por (data, tipo, texto)
pedidos_validacao	tuple[PedidoValidacao, ...]	lista	Ordenados pelo nome do piquete
piquetes	tuple[ResumoPiquete, ...]	lista	Um por piquete ativo, em ordem de nome

Movimentacao: “mover o lote X do piquete A para o B no dia D”

Campo	Tipo	Em JSON	Significado
id	UUID	texto	Id da movimentação; é o entidade_id do evento manejo_recomendado e o que vai nos botões do bot
data	date	"2026-09-28"	Dia da movimentação (sempre dia de manejo preferido)
lote_id / lote_nome	UUID / str	texto	Lote que muda
piquete_origem_id / piquete_origem_nome	UUID None / str None	texto ou null	De onde sai
piquete_destino_id / piquete_destino_nome	UUID / str	texto	Para onde vai
altura_destino_cm	float (1 casa)	número	Altura prevista do destino no início do dia
altura_entrada_alvo_cm	float	número	Altura ideal de entrada do capim do destino
altura_origem_cm	float None	número ou null	Altura prevista da origem no início do dia

Campo	Tipo	Em JSON	Significado
altura_saida_alvo_cm	float None	número ou null	Altura de saída do capim da origem
dias_previstos	int	número	Dias que o lote deve ficar no destino
motivo	str	texto	Frase pronta, em português, para o produtor
confianca	Confianca	"alta" "media" "baixa"	Confiança da recomendação
motivo_confianca	str	texto	Frase pronta: o dado mais fraco por trás da recomendação

Alerta

Campo	Tipo	Em JSON	Significado
tipo	TipoAlerta	um de: sem_piquete_apt, capacidade_excedida, aguardando_parametro, estimativa_indisponivel, continuo_acima_maxima, continuo_abaixo_minima, lote_sem_piquete, sem_dia_de_manejo	Tipo do aviso
data	date	"2026-10-01"	Dia a que se refere
texto	str	texto	Frase pronta para o produtor
confianca / motivo_confianca	Confianca / str	texto	Como na movimentação
piquete_id / lote_id	UUID None	texto ou null	A quem se refere

PedidoValidacao: pedido de medir o capim com régua

Campo	Tipo	Em JSON	Significado
piquete_id / piquete_nome	UUID / str	texto	Piquete a medir
motivo	str	texto	Frase pronta: por que o sistema está pedindo

ResumoPiquete: a fotografia de cada piquete (usada no mapa)

Campo	Tipo	Em JSON	Significado
piquete_id / nome	UUID / str	texto	Piquete
situacao	SituacaoPiquete	"ocupado" "descansando"	Tem gado ou está descansando
lote_atual_nome	str None	texto ou null	Lote que está nele
altura_hoje_cm	float None	número ou null	Altura estimada hoje; null = sem estimativa
altura_entrada_alvo_cm / altura_saida_alvo_cm	float None	número ou null	Alvos (nulos no contínuo ou sem parâmetro)

Campo	Tipo	Em JSON	Significado
confianca / motivo_confianca	Confianca / str	texto	Confiança da estimativa do piquete
faltantes	tuple[str, ...]	lista de texto	O que falta para estimar ou planejar; vazio = tudo certo

2. Stack e onde fica o seu código

Camada	Tecnologia
Backend	Python 3.12, FastAPI, psycopg 3 (SQL puro, sem ORM), Pydantic
Banco e login	Supabase (PostgreSQL + PostGIS + Auth)
Frontend	React + TypeScript + Vite, react-router, @supabase/supabase-js, CSS puro com variáveis
Hospedagem	Render (API) · Vercel (site) · GitHub Actions (ciclo semanal)

Arquivos que são seus (só você edita):

- Backend: `src/seugado/api/rotas_fazenda.py`, `rotas_lotes.py`, `rotas_plano.py` (já existem, vazios); `src/seugado/cadastro/fazenda.py`, `cadastro/lotes.py`; `src/seugado/jobs/ciclo.py`.
- Frontend: `frontend/src/paginas/Login.tsx`, `Onboarding.tsx`, `Configuracoes.tsx`, `Lotes.tsx`, `Plano.tsx` (já existem com texto provisório); `frontend/src/estilo/*`; componentes que você criar em `frontend/src/componentes/ui/`.
- Infra: `.github/workflows/ciclo-semanal.yml`, `render.yaml`.
- Testes: `tests/cadastro/test_fazenda.py`, `tests/cadastro/test_lotes.py`, `tests/jobs/*`.

Você usa, mas não edita: `api/deps.py` (Conexao), `api/auth.py` (Usuario, FazendaAutorizada, `fazenda_do_usuario`), `persistencia/*`, `contratos.py`, `planner/estado.py` (`peso_por_ua_kg`), `frontend/src/lib/*` (`api()`, `supabase`, `tipos`, `useFazenda()`), `frontend/src/App.tsx` (as rotas já apontam para as suas páginas), `frontend/src/componentes/Layout.tsx` (pode restilizar via `tema.css`). **Não adicione dependências.**

O Ezequiel também faz frontend (a página Mapa). Vocês dois usam o mesmo `tema.css`: mande a ele o link do Figma e as cores logo cedo.

3. Preparar o ambiente

Importante: o Python não lê o `.env` sozinho. Todo comando `uv run local` leva `--env-file .env` (como nos exemplos abaixo); sem isso a API responde erro 500 e os testes de banco são pulados.

1. Instale Git, **Node 20+** e **uv** (docs.astral.sh/uv). Clone e crie a branch `feat/leandro-fazenda-lotes-ciclo`.
2. Raiz: copie `.env.example` para `.env` e preencha com o que o Kauê mandar em privado: `DATABASE_URL`, `SUPABASE_URL`, `SUPABASE_ANON_KEY` e as duas do Earth Engine. **Nunca commite o**

`.env`.

3. frontend/.env a partir do `.env.example`: `VITE_SUPABASE_URL`, `VITE_SUPABASE_ANON_KEY`, `VITE_API_URL=http://localhost:8000`.
4. Backend: `uv sync --group dev` e `uv run --env-file .env uvicorn seugado.api.main:app --app-dir src --reload` (docs em `http://localhost:8000/docs`).
5. Frontend: `cd frontend && npm install && npm run dev` (`http://localhost:5173`).

Antes do PR: `uv run ruff check .`, `uv run --env-file .env pytest` e `npm run build` sem erros.

4. A regra de gravação do projeto

- **Fazenda** é configuração: INSERT/UPDATE direto na tabela `fazenda` (ADR-022).
- **Lote** (e as movimentações que você registra) é história: vira **evento**.

```
from datetime import UTC, datetime
from seugado.core.models import OrigemEvento, TipoEvento
from seugado.persistencia.eventos import registrar_evento
from seugado.persistencia.projecao_db import reconstruir_projecao

agora = datetime.now(UTC)
registrar_evento(conn, fazenda_id, TipoEvento.LOTE_CRIADO, OrigemEvento.PRODUTOR,
                 ocorrido_em=agora, payload=..., ator=str(usuario.id))
registrar_evento(conn, fazenda_id, TipoEvento.MANEJO_CONFIRMADO, OrigemEvento.PRODUTOR,
                 ocorrido_em=agora, payload=..., ator=str(usuario.id))
reconstruir_projecao(conn, fazenda_id)  # atualiza estado_lote / estado_piquete
conn.commit()                          # a ROTA faz o commit, uma vez, no fim
```

Importante: `ocorrido_em` é **sempre o momento atual** (`datetime.now(UTC)`). A data real do fato (“o lote está lá desde dia 20”) vai **dentro do payload** (`data_execucao`). A reconstrução processa os eventos pela ordem de `ocorrido_em`; se você datar a movimentação no passado, ela é processada antes de o lote existir e dá erro.

A conexão (`Conexao`) **não faz commit sozinha**. Para ler como dicionário, use `conn.cursor(row_factory=dict_row)` (`from psycopg.rows import dict_row`).

Tabelas que você usa:

Tabela	Uso
<code>fazenda</code>	<code>id</code> , <code>nome</code> , <code>timezone</code> , <code>funcionarios_disponiveis</code> , <code>manejos_por_funcionario_dia</code> , <code>dias_preferenciais_manejo</code> (<code>smallint[]</code>), <code>telegram_chat_id</code> , <code>codigo_vinculo_telegram</code> , <code>ativo</code>
<code>fazenda_usuario</code>	<code>usuario_id</code> (PK), <code>fazenda_id</code> : uma fazenda por usuário
<code>estado_lote</code> (derivada, só leitura)	<code>lote_id</code> , <code>nome</code> , <code>indissoluvel</code> , <code>composicao</code> (<code>jsonb</code>), <code>piquete_atual_id</code> , <code>desde</code> , <code>peso_vivo_total_kg</code>
<code>estado_piquete</code> (derivada, só leitura)	<code>piquete_id</code> , <code>nome</code> , <code>ativo</code> , <code>lote_atual_id</code>
<code>plano</code>	escrito pelo João; você só lê por <code>carregar_plano_atual</code>

Payloads dos eventos de lote (todos os campos obrigatórios):

```
# lote_criado
{"entidade_id": "<uuid do lote>", "nome": "Recria", "indissoluvel": False,
 "composicao": [{"categoria": "novilho", "n_animais": 150, "peso_medio_kg": 337.5,
                  "origem_peso": "ua_tabela"}]} # "produtor" | "ua_tabela"
# lote_alterado: os mesmos campos + "ativo": True
# lote_dissolvido
{"entidade_id": "<uuid do lote>"}
# manejo_confirmado (onde o lote está / movimentação avulsa)
{"entidade_id": "<uuid novo>", "lote_id": "<uuid do lote>",
 "piquete_destino_id": "<uuid do piquete>", "data_execucao": "2026-09-20"}
```

5. Contratos da sua parte

5.1 Rotas

Todas exigem login (Usuario). As que têm {fazenda_id} usam FazendaAutorizada (403 se a fazenda não for do usuário).

Método e caminho	Corpo	Resposta
GET /me	—	{"usuario_id", "email", "fazenda": Fazenda null}
POST /fazendas	FazendaIn	201 + Fazenda; 409 se o usuário já tem fazenda
PUT /fazendas/{fazenda_id}	FazendaIn	Fazenda
GET /fazendas/{fazenda_id}/teleg ram	—	{"vinculado": bool, "link": "https://t.me/<bot>?start=<codigo>"}
GET /fazendas/{fazenda_id}/lotes	—	Lote[] (por nome)
POST /fazendas/{fazenda_id}/lotes	LoteIn	201 + Lote
PUT /fazendas/{fazenda_id}/lotes /{lote_id}	LoteIn	Lote
DELETE /fazendas/{fazenda_id}/lotes /{lote_id}	—	204
GET /fazendas/{fazenda_id}/plano /atual	—	PlanoManejo ou 404
POST /fazendas/{fazenda_id}/ciclo	{"ingerir_satelite": true, "enviar": true}	PlanoManejo

```
// FazendaIn / Fazenda (Fazenda = FazendaIn + "id")
{ "nome": "Fazenda Boa Vista", "timezone": "America/Fortaleza",
  "funcionarios_disponiveis": 1, "manejos_por_funcionario_dia": 2,
  "dias_preferenciais_manejo": [0, 3] } // segunda e quinta

// LoteIn
```

```
{ "nome": "Recria", "indissoluvel": false,
  "composicao": [ {"categoria": "novilho", "n_animais": 150, "peso_medio_kg": null} ],
  "piquete_atual_id": "...",           // obrigatório
  "desde": "2026-09-25" }             // opcional; padrão: hoje

// Lote (resposta)
{ "id": "...", "nome": "Recria", "indissoluvel": false,
  "composicao": [ {"categoria": "novilho", "n_animais": 150, "peso_medio_kg": 337.5, "origem_peso":
"ua_tabela"} ],
  "piquete_atual_id": "...", "piquete_atual_nome": "Piquete 2", "desde": "2026-09-25",
  "peso_vivo_total_kg": 50625.0, "n_animais_total": 150 }
```

Os tipos TypeScript (Fazenda, Me, Lote, ComposicaoItem, PlanoManejo, ...) **já existem** em `frontend/src/lib/tipos.ts`. O `PlanoManejo` tem exemplo completo em `tests/fixtures/plano_exemplo.json`: use-o para montar a página do plano antes de a integração estar pronta.

5.2 Funções que o seu ciclo chama (de outras pessoas)

```
from seugado.delivery.confirmacao import confirmar_por_omissao # Leo
from seugado.sensing.ingestao import ingerir_leituras # Kauê
from seugado.persistencia.projecao_db import reconstruir_projecao # Kauê
from seugado.planner.carga import montar_estado_projetado # Kauê
from seugado.planner.otimizador import gerar_plano # João
from seugado.persistencia.planos import salvar_plano, carregar_plano_atual # João
from seugado.delivery.envio import enviar_plano # Leo

confirmar_por_omissao(conn, fazenda_id: UUID, hoje: date) -> int
ingerir_leituras(conn, fazenda_id: UUID, data_inicio: date, data_fim: date) -> int
reconstruir_projecao(conn, fazenda_id: UUID) -> EstadoFazenda
montar_estado_projetado(conn, fazenda_id: UUID, data_base: date) -> EstadoProjetado
gerar_plano(estado: EstadoProjetado, fazenda: Fazenda, agora: datetime, plano_id: UUID) ->
PlanoManejo
salvar_plano(conn, plano: PlanoManejo) -> None
enviar_plano(conn, plano: PlanoManejo, atualizado: bool = False) -> bool
```

Nenhuma delas faz commit: **o ciclo é que faz.**

5.3 Função que os outros chamam (sua)

```
# src/seugado/jobs/ciclo.py
def executar_ciclo(conn, fazenda_id: UUID, hoje: date | None = None,
                  ingerir_satelite: bool = True, enviar: bool = True,
                  atualizado: bool = False) -> PlanoManejo
```

O Leo chama `executar_ciclo(conn, fazenda_id, ingerir_satelite=False, atualizado=True)` quando o produtor responde “não fiz” ou “fiz diferente”. **Não mude essa assinatura.**

6. Tarefas (na ordem)

L0: GET /me e POST /fazendas primeiro, em PR pequeno (antes de tudo)

Por quê: o site inteiro depende de `GET /me`. Sem ele, o `RotaProtegida` não sabe quem está logado e manda todo mundo de volta para o login, inclusive o Ezequiel, que precisa da rota

`/mapa` funcionando. É a única tarefa da equipe que bloqueia outra pessoa.

Como: faça só a parte **backend** da L3 (`cadastro/fazenda.py` com `criar_fazenda` e `carregar_fazenda`, e as rotas `GET /me` e `POST /fazendas`). Abra um PR pequeno (`feat/leandro-me`) e **avise o Kauê para fazer o merge na hora**, sem esperar a segunda. Depois avise o Ezequiel para dar `git pull` na `main`. Enquanto a tela de onboarding não existe, cada pessoa cria a própria fazenda pelo `/docs` da API local (`POST /fazendas`, com o cabeçalho `authorization: Bearer <token>`; o token aparece no navegador, em DevTools → Application → Local Storage → a chave `sb-...-auth-token`, campo `access_token`).

Critérios de aceite:

- [] Logado e sem fazenda, o site vai para `/onboarding`; com fazenda, abre `/mapa`.
- [] Merge na `main` até as 10h.

L1: Identidade visual (Figma) + `tema.css` (timebox: 1 hora)

Por quê: na terça, a banca vai julgar também o produto. Uma paleta e uma tipografia consistentes nas 6 telas, incluindo o mapa do Ezequiel, fazem o sistema parecer um produto só.

Como:

1. No Figma, faça wireframes simples das 6 telas: Login, Onboarding, Mapa, Lotes, Plano da semana, Configurações. Foque em layout e hierarquia; é uma hora, não um design system.
2. Paleta (verde de pasto como primária; cores de estado para o mapa: verde, amarelo, azul, vermelho, cinza), tipografia (fonte do sistema) e raio de borda.
3. Passe para `frontend/src/estilo/tema.css`, trocando os valores das variáveis que já existem (`--cor-primaria`, `--cor-fundo`, ...) e acrescentando as de estado (`--cor-pronto`, `--cor-crescendo`, `--cor-ocupado`, `--cor-sair`, `--cor-sem-estimativa`).
4. Mande o link do Figma e as variáveis ao Ezequiel **antes das 10h**.

Critérios de aceite:

- [] 6 telas no Figma e link compartilhado com a equipe.
- [] `tema.css` com as variáveis; nenhuma cor escrita direto nas páginas.

Fora do escopo: logo definitivo; biblioteca de componentes; modo escuro.

L2: Login (`Login.tsx`)

Por quê: cada produtor vê só a própria fazenda. O login é do Supabase: você não guarda senha.

Como:

1. Formulário com e-mail e senha e dois modos: **Entrar** (`supabase.auth.signInWithPassword`) e **Criar conta** (`supabase.auth.signUp`).
2. Sucesso → `navigate("/mapa")`. O `RotaProtegida` já redireciona para `/onboarding` se ainda não houver fazenda.

3. Erros do Supabase em português (“E-mail ou senha incorretos”, “Senha precisa de 6 caracteres”).
4. **No painel do Supabase** (combine com o Kauê, que é o dono do projeto):
 - *Authentication* → *Email*: “Confirm email” desligado;
 - *URL Configuration*: Site URL = endereço do Vercel;
 - Redirect URLs com `http://localhost:5173`.

Critérios de aceite:

- [] Criar conta → entra direto → cai no onboarding.
- [] Sair e entrar de novo funciona.
- [] Erro aparece na tela, sem `alert()`.

Fora do escopo: login social; recuperação de senha; perfis e permissões.

L3: Fazenda: backend, onboarding e configurações

Por quê: a fazenda guarda a **rotina** que o otimizador respeita: quantos funcionários, quantas movimentações cada um faz por dia e em quais dias da semana se mexe no gado. É o diferencial do SeuGado (R7 e R8 do motor).

Como (backend, `cadastro/fazenda.py` + `rotas_fazenda.py`):

1. `criar_fazenda(conn, usuario_id, dados)` -> UUID: se `fazenda_do_usuario` já existir → erro 409. Senão, `INSERT INTO fazenda (nome, timezone, funcionarios_disponiveis, manejos_por_funcionario_dia, dias_preferenciais_manejo) ... RETURNING id` e `INSERT INTO fazenda_usuario`.
2. `carregar_fazenda(conn, fazenda_id)` -> Fazenda (a dataclass `Fazenda` de `seugado.core.models`, com `dias_preferenciais_manejo` como `tuple[int, ...]`). O ciclo usa esta função.
3. `atualizar_fazenda(conn, fazenda_id, dados)`: `UPDATE`.
4. Validação (Pydantic): `nome` não vazio; `funcionarios_disponiveis` ≥ 1 ; `manejos_por_funcionario_dia` ≥ 1 ; `dias_preferenciais_manejo` com pelo menos 1 valor, todos entre 0 e 6, sem repetição (ordene); `timezone` padrão `America/Fortaleza`.
5. GET `/me`: `usuario_atual` + `fazenda_do_usuario` + `carregar_fazenda`.
6. GET `/fazendas/{id}/telegram`: `link = f"https://t.me/{TELEGRAM_BOT_USERNAME}?start={codigo_vinculo_telegram}"` (variável de ambiente), `vinculado = telegram_chat_id is not None`.
7. Cada rota que escreve termina com `conn.commit()`.

Como (frontend):

- **Onboarding:**
 - boas-vindas + nome da fazenda;
 - “Quantas pessoas cuidam do manejo do gado?”;
 - “Quantos lotes cada pessoa consegue mudar de piquete por dia?”;
 - “Em quais dias da semana vocês costumam mexer no gado?”, com 7 caixas seg...dom e segunda e quinta marcadas por padrão.

Salvar → POST /fazendas → recarregar() (de useFazenda) → /mapa com a mensagem “Agora desenhe seus piquetes.”

- **Configurações:** o mesmo formulário (PUT) + cartão **Telegram**:

- se não vinculado: “Abra este link no celular e toque em **Começar**”, com o link e um botão “Já abri”, que recarrega o status;
- se vinculado: “Telegram conectado ✓”.

Critérios de aceite:

- [] Usuário novo passa pelo onboarding e não volta mais a ele.
- [] Mudar os dias de manejo persiste.
- [] GET /me sem fazenda devolve "fazenda": null.

Fora do escopo: mais de uma fazenda por usuário; convidar outros usuários; endereço ou município.

L4: Lotes: backend e página

Por quê: o consumo do lote é metade da conta do otimizador: quanto o lote come define em quantos dias ele rapa o piquete. E o otimizador precisa saber **onde cada lote está agora**.

Como (backend, cadastro/lotes.py + rotas_lotes.py):

1. `criar_lote(conn, fazenda_id, ator, nome, indissoluvél, composicao, piquete_atual_id, desde) -> UUID:`
 - valide:
 - nome não repetido entre os lotes da fazenda (`estado_lote`);
 - composição com pelo menos 1 item, categorias sem repetição, `n_animais >= 1`;
 - `peso_medio_kg`, se informado, entre 20 e 1.500;
 - o piquete existe, está ativo e **está vazio** (`estado_piquete.lote_atual_id IS NULL`);
 - `desde <= hoje`;
 - peso ausente → `peso_por_ua_kg(categoria)` (de `seugado.planner.estado`) com `origem_peso: "ua_tabela"`; informado → `"produtor"`;
 - registre `LOTE_CRIADO` e em seguida `MANEJO_CONFIRMADO` (`NOVO entidade_id, lote_id, piquete_destino_id = piquete_atual_id, data_execucao = desde`), ambos com `ocorrido_em = agora`;
 - `reconstruir_projecao`.
2. `editar_lote(...)`: registre `LOTE_ALTERADO` (todos os campos + `ativo: True`). Se o `piquete_atual_id` mudou, registre também `MANEJO_CONFIRMADO` para o novo piquete (vazio) com `data_execucao = hoje`: é uma correção de posição feita pela web.
3. `dissolver_lote(...)`: `LOTE DISSOLVIDO`. O piquete onde ele estava passa a descansar sozinho na reconstrução.
4. `listar_lotes(conn, fazenda_id)`: `estado_lote` + nome do piquete (`estado_piquete`); `n_animais_total` = soma da composição.
5. Rotas: `ValueError/ValidationError` → 400, lote inexistente → 404, piquete ocupado → 409. `conn.commit()` no fim.

Como (frontend, `Lotes.tsx`):

- Lista em cartões:

Recria · 150 animais · Piquete 2 desde 25/09 · 50,6 t de peso vivo

- Formulário “Novo lote / Editar”:
 - **Nome**;
 - **Composição**: linhas com categoria (`select`), quantidade e peso médio (kg), com a dica “Não sabe? Deixe em branco: estimamos pela tabela de Unidade Animal (bezerro 112,5 · novilho 337,5 · adulto 450 kg)”, e o botão “+ categoria”;
 - **Onde o lote está agora**: `select` com os piquetes livres, de `GET /fazendas/{id}/piquetes` (rota do Ezequiel), mais o piquete atual do lote, na edição;
 - **Desde** (data);
 - **Lote indissolúvel**, com a explicação “Marque se este lote nunca pode ser juntado a outro, por exemplo vacas com bezerro ao pé.”
- Se ainda não há piquete, mostre “Cadastre os piquetes no Mapa antes dos lotes”, com link.
- Dissolver com confirmação na própria tela.

Critérios de aceite:

- [] Criar “Recria: 150 novilhos, sem peso, no Piquete 2” grava peso 337,5 com `origem_peso: "ua_tabela"` e o Piquete 2 aparece ocupado.
- [] Não deixa colocar dois lotes no mesmo piquete.
- [] Editar a composição atualiza o peso vivo total.

Fora do escopo: raça, sexo, brinco, pesagem individual (ERP, ADR-001); fusão de lotes (F-022); histórico de movimentações na tela.

L5: Plano da semana (`rotas_plano.py` + `Plano.tsx`)

Por quê: o Telegram é o canal do dia a dia, mas é na web que se apresenta, se confere e se dispara o plano. Na terça, esta página é a demo.

Como (backend):

1. `GET /fazendas/{id}/plano/atual`: `carregar_plano_atual(conn, fazenda_id) → plano_para_dict`, OU 404 “Ainda não há plano”.
2. `POST /fazendas/{id}/ciclo`: chama `executar_ciclo(conn, fazenda_id, ingerir_satelite=corpo.ingerir_satelite, enviar=corpo.enviar)` e devolve o plano. A chamada pode levar até ~1-2 min (satélite + clima). Erros de serviço externo → 502 com a mensagem.

Como (frontend, `Plano.tsx`):

- Cabeçalho: “Plano de seg 28/09 a dom 04/10” · “gerado em 28/09 às 05:00” · botão **Gerar plano agora**, com spinner, desabilitado durante a chamada e o texto “Buscando satélite e clima...”.
- **Movimentações**, agrupadas por dia (só os dias com movimento). Em cada cartão:
 - título: **Mover Recria: Piquete 2 → Piquete 1**;
 - `motivo`;
 - “Previsão: 3 dias no piquete”;
 - selo de confiança (alta/média/baixa, com cor) e `motivo_confianca` em cinza.
- **Alertas**, cada um com ícone por `tipo` e o `texto`.
- **Medições pedidas** (`pedidos_validacao`): “Meça a altura do **Piquete 4** com uma régua”, com o motivo e um campo “cm” + botão que chama `POST /fazendas/{id}/piquetes/{piquete_id}/alturas` (rota do Ezequiel).
- Sem plano (404): estado vazio com o botão “Gerar o primeiro plano”.
- **Não recalcule nada no frontend**: mostre exatamente o que vem do plano.

Critérios de aceite:

- [] Com `plano_exemplo.json`, a página mostra 2 dias (segunda com 2 movimentos, quinta com 1), 3 alertas e 1 pedido de medição.
- [] “Gerar plano agora” atualiza a página com o plano novo.

Fora do escopo: editar o plano; aceitar ou recusar pela web (isso é pelo Telegram, com o Leo); histórico de planos.

L6: Ciclo semanal (`jobs/ciclo.py`)

Módulos que ainda não estão na sua branch. `gerar_plano/salvar_plano/carregar_plano_atual` (João) e `confirmar_por_omissao/enviar_plano` (Leo) só chegam na `main` na integração de segunda. Por isso, **importe-os dentro da função** (`from seugado.planner.otimizador import gerar_plano` na primeira linha de `executar_ciclo`, e o mesmo em `rotas_plano.py`), e nos testes injete módulos falsos com `monkeypatch.setitem(sys.modules, "seugado.planner.otimizador", modulo_falso)`. O `projecao_db` do Kauê já está na `main`; `ingestao` e `carga` entram ao longo de domingo. Se ainda não estiverem quando você chegar aqui, use a mesma técnica para eles.

Por quê: é o que faz o SeuGado rodar **sozinho** toda segunda-feira. É a definição de MVP completo (F-015).

Como:

```
def executar_ciclo(conn, fazenda_id, hoje=None, ingerir_satelite=True, enviar=True,
atualizado=False):
    agora = datetime.now(UTC)
    fazenda = carregar_fazenda(conn, fazenda_id)
    hoje = hoje or agora.astimezone(ZoneInfo(fazenda.timezone)).date()
    confirmar_por_omissao(conn, fazenda_id, hoje) # 1
    if ingerir_satelite:
        ingerir_leituras(conn, fazenda_id, hoje - timedelta(days=30), hoje) # 2
    reconstruir_projecao(conn, fazenda_id) # 3
    conn.commit()
    estado = montar_estado_projetado(conn, fazenda_id, hoje) # 4
    plano = gerar_plano(estado, fazenda, agora, uuid4()) # 5
    salvar_plano(conn, plano) # 6
```

```

conn.commit()
if enviar:
    enviar_plano(conn, plano, atualizado=atualizado)
return plano
# 7

```

1. Exatamente nessa ordem (ADR-024). Os commits intermediários garantem que satélite e respostas fiquem salvos mesmo se o plano falhar depois.
2. `executar_todas()`: abre a própria conexão (`psycopg.connect(os.environ["DATABASE_URL"])`) e percorre as fazendas `ativo` que têm pelo menos 1 piquete ativo. Cada fazenda roda num `try`: erro → `rollback()`, imprime o erro com o id da fazenda e **continua** com a próxima. No fim, código de saída 1 se alguma falhou (o GitHub mostra em vermelho).
3. Linha de comando (`python -m seugado.jobs.ciclo`, com `argparse`): `--todas`, OU `--fazenda <uuid>`, mais `--sem-satelite` e `--sem-envio`.
4. Testes: troque as sete funções por falsas (`monkeypatch`) e confira a **ordem** das chamadas, os commits e que uma fazenda com erro não derruba as outras.

Critérios de aceite:

- [] Ordem das chamadas exatamente 1→7.
- [] `--sem-satelite` não chama o Earth Engine.
- [] Uma fazenda com erro não impede as outras.

Fora do escopo: fila, paralelismo, agendamento dentro da API, reenvio automático.

L7: Deploy

Por quê: “funcionando de verdade” na terça quer dizer acessível pela internet e rodando sem o computador de ninguém ligado.

Como:

1. **API no Render** (plano gratuito), *New* → *Web Service* a partir do repositório GitHub:
 - Runtime Python; variável `PYTHON_VERSION=3.12`;
 - Build: `pip install uv && uv sync --frozen --no-dev`;
 - Start: `uv run uvicorn seugado.api.main:app --app-dir src --host 0.0.0.0 --port $PORT`;
 - Health check: `/saude`;
 - Variáveis: `DATABASE_URL`, `SUPABASE_URL`, `SUPABASE_ANON_KEY`, `SEUGADO_CORS_ORIGINS=https://<seu-app>.vercel.app,http://localhost:5173`, `SEUGADO_GEE_PROJECT`, `SEUGADO_GEE_SERVICE_ACCOUNT_JSON`, `TELEGRAM_BOT_TOKEN`, `TELEGRAM_BOT_USERNAME`, `TELEGRAM_WEBHOOK_SECRET` (as três últimas vêm do Leo).
 - Guarde a configuração em `render.yaml` na raiz.
 - O plano grátis dorme após 15 min sem uso, e a primeira chamada demora ~1 min.

Antes da apresentação, abra `/saude` para acordar a API.
2. **Site no Vercel:** *Add New* → *Project*, root directory `frontend`, framework Vite; variáveis `VITE_SUPABASE_URL`, `VITE_SUPABASE_ANON_KEY`, `VITE_API_URL=https://<api>.onrender.com`. O `vercel.json` (rotas do SPA) já existe.
3. **Ciclo no GitHub Actions**, `.github/workflows/ciclo-semanal.yml`:
 - `on.schedule.cron: "0 8 * * 1"` (08:00 UTC = segunda 05:00 em Fortaleza) e `workflow_dispatch` com os inputs `fazenda_id` (texto, opcional) e `sem_satelite` (booleano);

- passos: `actions/checkout`, `astral-sh/setup-uv` (versão estável atual), `uv sync --frozen --no-dev`, `uv run python -m seugado.jobs.ciclo --todas` (ou `--fazenda` se o input vier), com `PYTHONPATH: src;`
- secrets do repositório: `DATABASE_URL`, `SEUGADO_GEE_PROJECT`, `SEUGADO_GEE_SERVICE_ACCOUNT_JSON`, `TELEGRAM_BOT_TOKEN`.

4. Depois do deploy, mande à equipe as URLs do site e da API. O Leo precisa da URL da API para registrar o webhook do Telegram.

Critérios de aceite:

- [] O site no Vercel faz login e fala com a API no Render.
- [] “Run workflow” no GitHub roda o ciclo e o plano aparece no site.
- [] Nenhuma chave aparece no repositório.

Fora do escopo: domínio próprio; ambientes separados de homologação e produção; monitoramento.

7. Integração na segunda (a sua parte é a cola)

1. Merge na ordem: Ezequiel → você (fazenda e lotes) → João → Leo → você (ciclo e deploy).
2. Com uma fazenda de teste real, na web: login → onboarding → piquetes (Ezequiel) → lotes → “Gerar plano agora” → o plano aparece aqui, no mapa e no Telegram do Leo.

8. Usando IA no seu fluxo

Dê à IA este documento e o arquivo que ela vai editar, **uma tarefa por vez**. Exemplo:

“Leia docs/equipe/LEANDRO.md, seções 4, 5 e a tarefa L4. Implemente `src/seugado/cadastro/lotas.py` e as rotas em `src/seugado/api/rotas_lotas.py` seguindo exatamente os payloads e regras descritos. Use `registrar_evento` e `reconstruir_projecao` já existentes. Não altere outros arquivos nem adicione dependências.”

Se a IA quiser mudar arquivo de outra pessoa, instalar pacote ou alterar contrato, a resposta é não: fale com o Kauê.